IMPERIAL

pyop3:
An Execution Abstraction for Compact Computational Kernels on Unstructured Meshes

Connor J. Ward
David A. Ham
Jack D. Betteridge

c.ward20@imperial.ac.uk

Mesh data ≠ ND arrays

N-dimensional (ND) arrays are the backbone of many areas of computational science. In particular they underpin machine learning frameworks such as PyTorch and JAX.

Their popularity as an abstraction is due to two factors:

1. It is a very natural programming model for many domains.
2. Tensor operations may be compiled to extremely high performance code.

Unfortunately, ND arrays are not a suitable fit for calculations that involve unstructured meshes:

1. Mesh data can be associated with different topological entities (e.g. cells and/or vertices, fig. 1). This results in 'ragged' dimensions in the data structure (fig. 2) that cannot be represented using typical ND arrays.
2. Mesh data are distributed in parallel and have complicated processes for ensuring global correctness.

pyop3 is this missing abstraction. It is a library that provides a numpy-like interface for unstructured mesh data that is compiled to fast code. Just like PyTorch and JAX, pyop3 will allow users to easily develop complex and performant numerical methods by composing operations at a convenient level of abstraction.

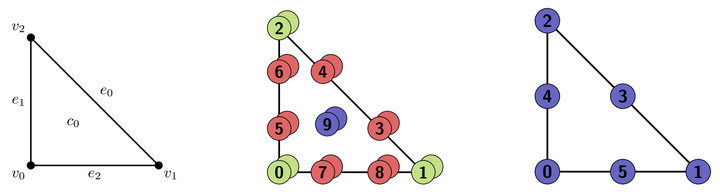

Figure 1: The Scott-Vogelius finite element pair $[P_3]^2 \oplus P_2^{\text{disc}}$ (middle and right) on a reference triangle (left). DoFs associated with vertices, edges and cells are shown in green, red and blue respectively. P_2^{disc} (right) is discontinuous, so all the DoFs are considered to belong to the cell.

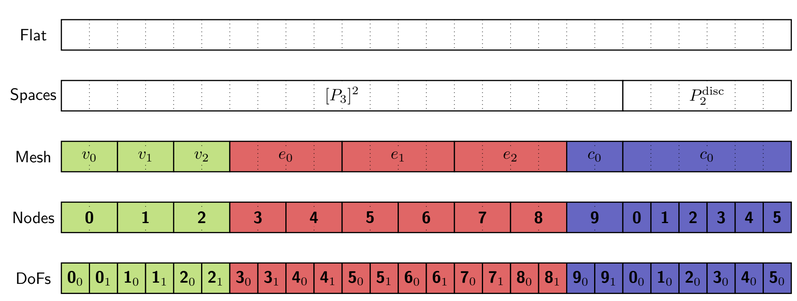

Figure 2: The data layout structure for a one-cell mesh of the finite element pair in fig. 1. It is not possible to capture all of this detail using an ND array.

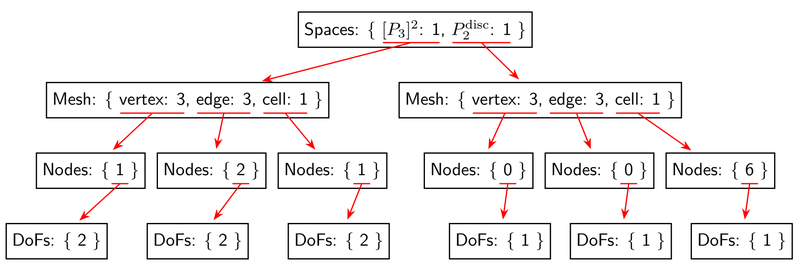

Figure 3: Graphical representation of an axis tree that describes the data layout shown in fig. 2. Unlike ND arrays, all information is captured by the abstraction.

The pyop3 language

pyop3 is a domain-specific language (DSL) that is embedded in Python. Users express operations using a numpy-like syntax (fig. 4) that are just-in-time compiled to high-performance code (fig. 5).

```
cell = mesh.cells.iter()
vec[closure(cell)][0][::2]
```

Figure 4: A non-trivial example of pyop3 numpy-like syntax in action: accessing the even numbered vertices around a cell. `closure(cell)` returns all entities that touch the cell and indexing with `0` extracts only the vertices. Note both the similarity between this code and typical ND array usage with numpy, and also how indexing operations may be chained together.

PYTHON

```
loop(cell := mesh.cells.iter(),
     kernel(vec1[closure(cell)], vec2[cell]))
```

CODE GENERATION

C

```
void pyop3_kernel(array_0, array_1, array_2) {
  double t_0[3];
  double t_1[1];
  for (int32_t i_0 = 0; i_0 < ncells; ++i_0) {
    for (int32_t i_1 = 0; i_1 < 3; ++i_1)
      t_0[i_1] = array_0[array_1[3 * i_0 + i_1]];
    t_1[0] = 0.0;
    kernel(&(t_0[0]), &(t_1[0]));
    array_2[i_0] = array_2[i_0] + t_1[0];
  }
}
```

Figure 5: Simplified example of the code generated from a pyop3 loop expression.

Operations in pyop3 are composable, and so it is possible to express non-trivial looping operations such as that shown in fig. 6.

```
loop(vertex := mesh.vertices.iter(),
     loop(cell := star(vertex, k=2).iter(),
          kernel(in_vec[closure(cell)], out_vec[vertex])))
```

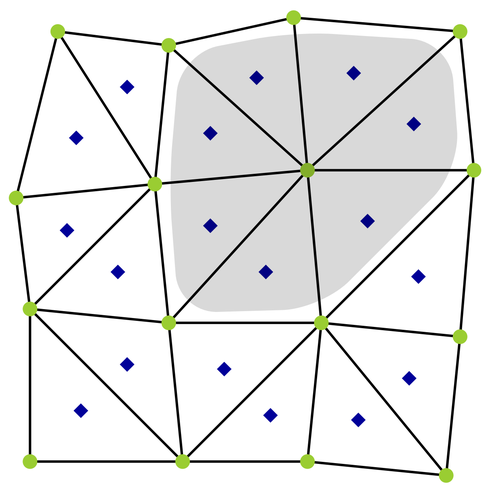

Figure 6: pyop3 loop expression describing an operation over a vertex patch. `star(vertex, k=2)` will return all cells that touch the current vertex.

Representing mesh data

To represent mesh data without losing important topological information, pyop3 necessarily has a much richer abstraction for describing unstructured data layouts than simple ND arrays: the axis tree (fig. 3).

4. GPU support

- Just like many machine learning frameworks, pyop3 is agnostic to the underlying hardware and in principle can generate code to execute on arbitrary platforms.
- Preliminary support for NVIDIA GPUs has already been merged.
- Next steps are... (cite MLIR)

4. Future work

Once pyop3 has been fully integrated into Firedrake there is scope for a variety of performance improvements and avenues of research to pursue. Possible low-level changes include:

- Data layout transformations
- Loop tiling

Whilst at a high level the following should also become expressible:

- Physics-based preconditioners
- Slope limiters
- Adaptivity
- Particle methods
- And more!